

Challenge: IMDb Lakehouse to OLAP Pipeline

On a daily basis, Teleparty users are generating large amounts of data daily. We receive a variety of event driven data from our users. The data gives us viewership insights into the type of shows our users are watching and the length of time spent in a watch party.

The purpose of this exercise is to understand how you as a data engineer would go about ingesting a small subset of data, creating an appropriate database schema for this data and preparing corresponding data queries to answer a variety of questions about incoming data.

Context

Teleparty needs to move beyond static data processing. Our analysts require sub-second query performance on hundreds of millions of rows. We are moving toward an architecture where **PySpark** handles the heavy lifting (cleansing/partitioning), and an **OLAP engine** serves the final insights.

Objective

Build a local data pipeline that:

1. Downloads the 2GB IMDb dataset from Kaggle (login required) - <https://www.kaggle.com/datasets/ashirwadsangwan/imdb-dataset>
 - a. Extract the necessary data for analysis, such as:
 - i. Movie titles, ratings, episodes and other pertinent information.
2. Processes the IMDb dataset using **PySpark**.
3. Saves the data as partitioned **Parquet** files (the "Lake").
4. Ingests or mounts that data into an **OLAP engine** of your choice for high-speed analytics.

Requirements

1. **The Environment:**
 - Use **Docker Compose** to spin up a Spark cluster (Master/Worker).
 - Ensure your environment can run your OLAP of choice locally.
2. **Transformation (PySpark):**
 - Clean and transform the IMDb dataset (titles, ratings, and episodes).
 - Export the result as **Snappy-compressed Parquet** files.
 - **Crucial:** Apply a partitioning strategy that makes sense for time-series or category-based analysis.
3. **OLAP Integration (The "Load" Step):**
 - Load the Parquet data into your chosen OLAP engine.
4. **Analytics Layer:**

- Demonstrate that the OLAP engine is significantly faster than raw Spark for these queries.

Deliverables

- **Infrastructure:** A `docker-compose.yml` that orchestrates Spark and your OLAP of choice (if applicable).
- **The Pipeline:** A PySpark script (`etl_job.py`) and a loading script (`load_to_olap.py`).
- **The Schema:** DDL (Data Definition Language) files for your OLAP tables, including indexes or primary keys.
- **Performance Note:** A brief section in your README explaining why you chose your OLAP engine of choice.
- A Github repo containing all code implemented to complete this data challenge.

Notes

Use of AI/LLMs is permitted; however:

- You must be able to explain all code generated without use of AI
 - Please include a `PROMPTS.md` file (markdown formatted) containing all the prompts used including your choice of LLM.